Understanding Pod Phases & Lifecycle in Kubernetes

A practical guide for DevOps engineers and Kubernetes operators on how pods move through their lifecycle — from scheduling to termination — and how to use that knowledge to troubleshoot and manage workloads effectively.

KUBERNETES | POD LIFECYCLE | DEVOPS

What Is a Pod in Kubernetes?

The Smallest Deployable Unit

A **pod** is the atomic unit of deployment in Kubernetes. It wraps one or more containers together with shared networking and storage, providing a cohesive execution environment.

Why Pods Matter

Every workload in Kubernetes — whether it's a web server, a batch job, or a sidecar proxy — runs inside a pod. Understanding how pods behave across their lifecycle is foundational to:

- Diagnosing scheduling failures
- Debugging crash loops and restarts
- Optimizing resource allocation
- Writing reliable health checks and probes

The Five Pod Phases at a Glance

Kubernetes assigns every pod a **phase** — a high-level summary of where the pod currently sits in its lifecycle. There are five officially defined phases.

⏳ Pending

Accepted by the cluster, but containers haven't started yet.

🟢 Running

At least one container is actively running or being restarted.

✅ Succeeded

All containers exited cleanly with code 0. No restarts will occur.

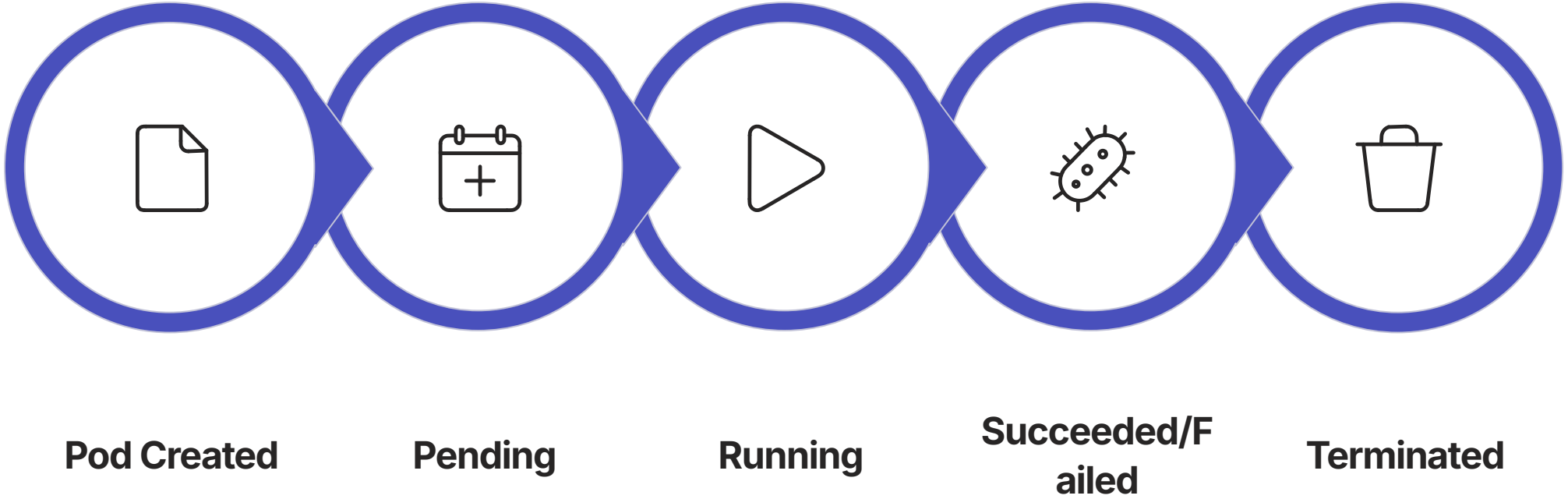
❌ Failed

One or more containers exited with a non-zero code or were terminated by the system.

❓ Unknown

The node hosting the pod is unreachable; state cannot be determined.

Pod Lifecycle: End-to-End Flow



From the moment a pod manifest is submitted to the API server until the pod is garbage-collected, every transition is deterministic and observable. Knowing this flow lets you intercept problems at exactly the right stage — whether that's a scheduling bottleneck, a container crash, or a graceful shutdown failure.

Phase 1: Pending

The **Pending** phase begins the moment the Kubernetes API server accepts the pod definition. The scheduler then evaluates all available nodes and attempts to bind the pod. Common reasons a pod stays Pending include **insufficient CPU or memory**, missing node selectors, taints without matching tolerations, or a PersistentVolumeClaim that hasn't been bound yet.

Pending Phase — Hands-On Scenario

The following manifest requests **10Gi of memory and 4 CPUs** — resources most typical development clusters cannot satisfy on a single node. Applying it causes the pod to remain in Pending indefinitely.

```yaml
# pending-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pending-pod
spec:
  containers:
  - name: busybox
    image: busybox
    resources:
      requests:
        memory: "10Gi"
        cpu: "4"
    command: ["sleep", "3600"]
```

Apply & Inspect

`kubectl apply -f pending-pod.yaml`
Then check status with:
`kubectl describe pod pending-pod`

What to Look For

In the `Events` section of `kubectl describe`, you will see a `FailedScheduling` event indicating that no nodes matched the resource request. This is your primary diagnostic signal for Pending pods.

Diagnosing Pending Pods

Use these commands and signals to quickly narrow down why a pod is stuck in Pending.

Check Pod Events

`kubectl describe pod <pod-name>` — look for `FailedScheduling` or `Unschedulable` in the Events section. This is the fastest way to identify scheduling failures.

Inspect Node Capacity

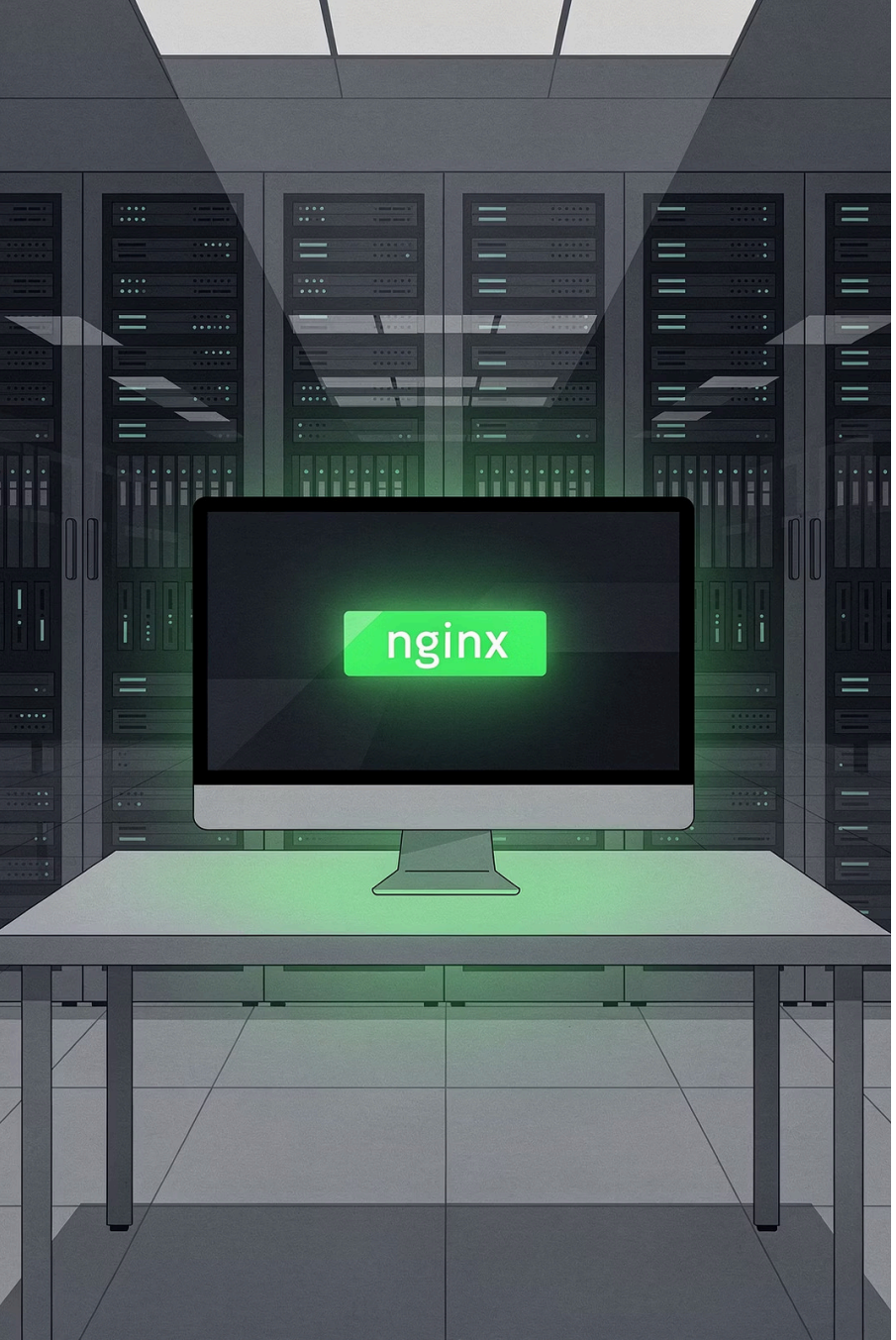
`kubectl describe nodes` — compare the **Allocatable** resources against what the pod requests. If all nodes are over-committed, you need to scale the cluster.

Check Taints & Tolerations

`kubectl get nodes -o json | jq '.items[].spec.taints'` — verify that the pod's tolerations match node taints. A single missing toleration blocks all scheduling.

PVC Binding Status

`kubectl get pvc` — if a volume claim shows `Pending`, the pod won't be scheduled until the claim is bound to a PersistentVolume.



Phase 2: Running

A pod enters the **Running** phase once it has been bound to a node and at least one of its containers is actively executing. "Running" does not guarantee the application is healthy — it simply means the container process has started. Containers within a Running pod can individually be in `waiting`, `running`, or `terminated` states, which you can inspect independently via `kubectl get pod -o json`.

Running Phase — Hands-On Scenario

This minimal manifest deploys an **Nginx web server**. On a healthy cluster with available resources, this pod transitions to Running within seconds of being applied.

```
# running-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: running-pod
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

Apply & Watch

`kubectl apply -f running-pod.yaml`
Monitor transitions live:
`kubectl get pod running-pod -w`

Verify Container State

Once Running, confirm the container is healthy:
`kubectl describe pod running-pod`
Look for `State: Running` and a recent `Started` timestamp under the container details section.

Container States Inside a Running Pod

A pod in the Running phase can contain containers in three distinct sub-states. Understanding this distinction is essential for diagnosing partial failures.

Running

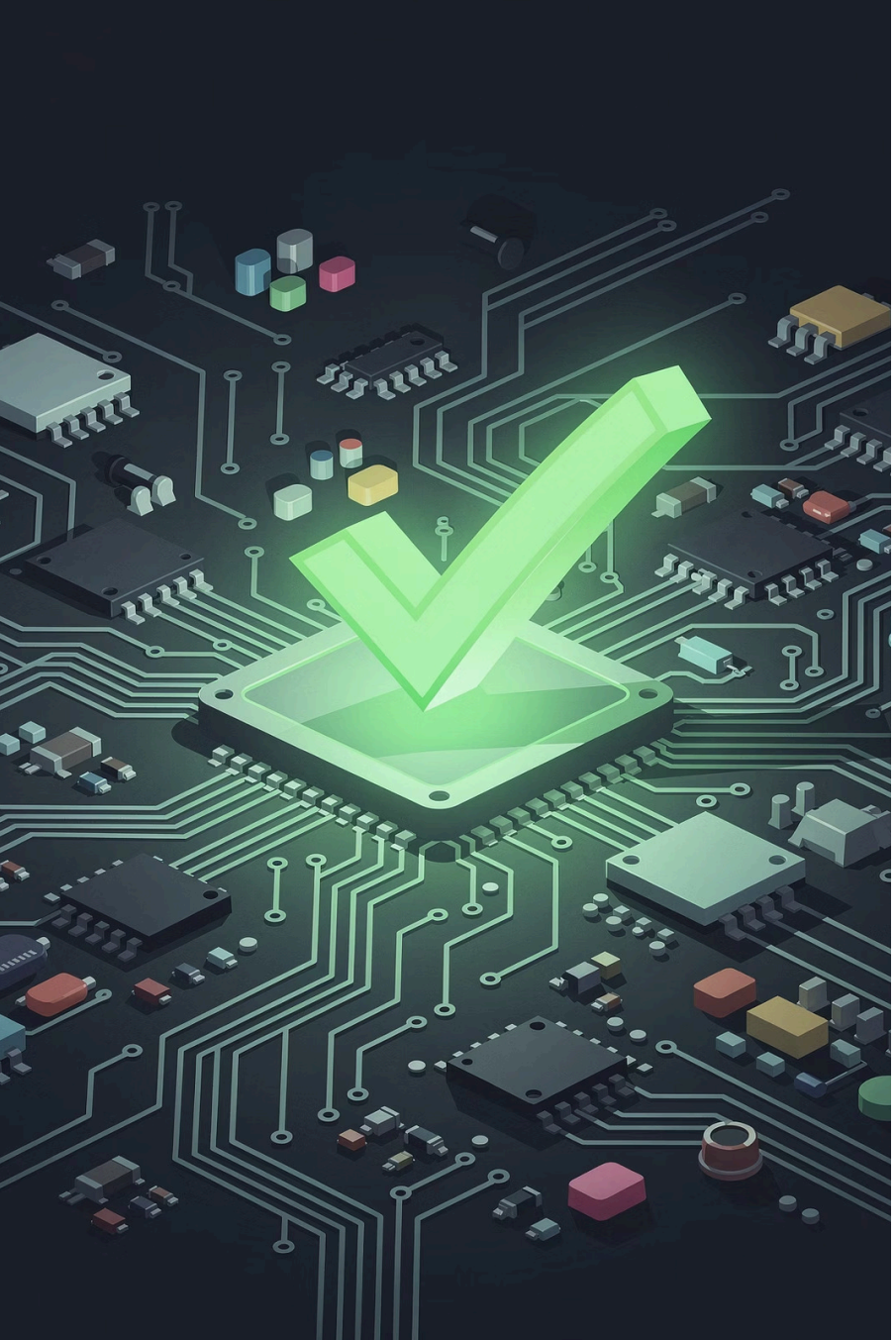
The container process is actively executing without issues. The `startedAt` timestamp is set and the process PID is live on the node.

Waiting

The container has not yet started. Common reasons: pulling the image (`ContainerCreating`), waiting for an init container, or a `CrashLoopBackOff` back-off period.

Terminated

The container's process has exited. The exit code and reason are recorded. A terminated container inside a Running pod indicates a partial failure that may trigger a restart.



Phase 3: Succeeded

A pod reaches the **Succeeded** phase when *every* container in the pod has exited with an exit code of `0` and the restart policy prevents any further restarts. This phase is terminal — the pod will not be rescheduled or restarted. It is most commonly observed in **batch jobs**, **data migration scripts**, and **one-time initialization tasks** managed by Kubernetes `Job` resources.

Succeeded Phase — Hands-On Scenario

This pod runs a single `echo` command. Since the process exits immediately with code 0 and the default restart policy is `Never` (for pods not managed by a controller), it will transition directly to Succeeded.

```
# succeeded-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: succeeded-pod
spec:
  restartPolicy: Never
  containers:
  - name: busybox
    image: busybox
    command: ["echo", "Hello, Kubernetes!"]
```

Apply & Confirm

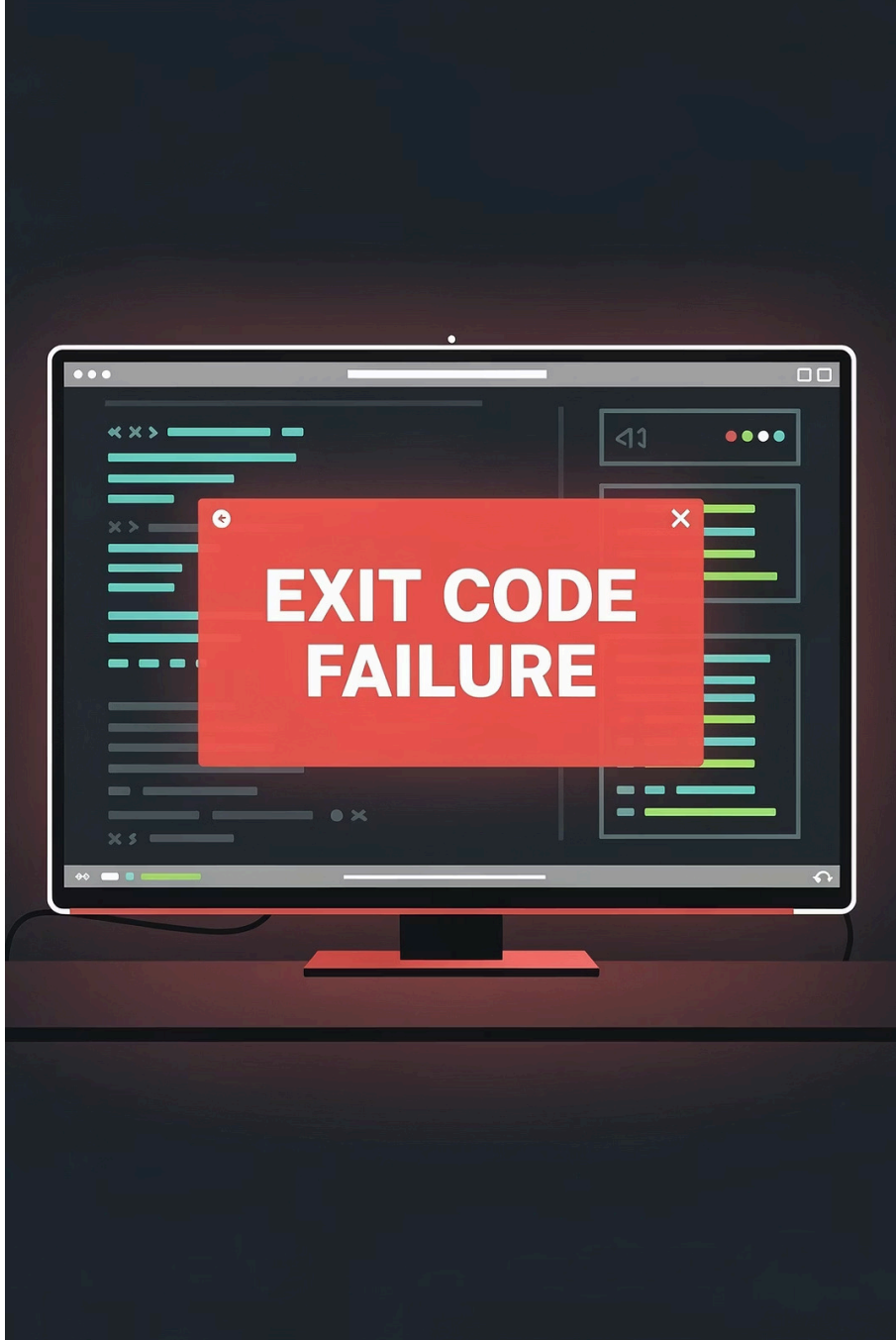
`kubectl apply -f succeeded-pod.yaml`
Check the final status:
`kubectl get pod succeeded-pod`

Inspect Logs

Even after a pod succeeds, logs are preserved:
`kubectl logs succeeded-pod`
Output: `Hello, Kubernetes!`
This confirms the container executed and exited cleanly.

Phase 4: Failed

A pod enters the **Failed** phase when at least one container exits with a **non-zero exit code**, or when a container is forcibly terminated by the system (e.g., OOMKilled). Like Succeeded, Failed is a terminal phase — no restarts occur when the restart policy is set to `Never` or `OnFailure` has been exhausted. The failure reason and exit code are preserved for post-mortem inspection.



Failed Phase — Hands-On Scenario & Diagnosis

```yaml
# failed-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: failed-pod
spec:
  restartPolicy: Never
  containers:
  - name: busybox
    image: busybox
    command: ["sh", "-c", "exit 1"]  # Deliberately exit non-zero
```

1

Apply the manifest

`kubectl apply -f failed-pod.yaml`

2

Check pod phase

`kubectl get pod failed-pod` — STATUS will show `Error`

3

Inspect exit code

`kubectl describe pod failed-pod` — look for `Exit Code: 1` under container state

4

Read logs

`kubectl logs failed-pod` — capture any error output before cleanup

Common Failure Reasons & Exit Codes

Understanding exit codes is the fastest path to root cause analysis when a pod enters the Failed phase.

Exit Code	Signal / Reason	Common Cause
`0`	Success	Process completed normally — Succeeded phase
`1`	General Error	Application error, misconfiguration, or missing file
`137`	SIGKILL (OOMKilled)	Container exceeded its memory limit; kernel killed it
`139`	SIGSEGV	Segmentation fault — bug in application code
`143`	SIGTERM	Graceful termination signal sent by Kubernetes
`255`	Unknown	Container entry point could not be executed

Phase 5: Unknown

What It Means

The **Unknown** phase is declared by the control plane when the **node hosting the pod becomes unreachable** — typically due to a network partition, a node crash, or the `kubelet` process dying on that node. The API server cannot retrieve pod status, so it defaults to Unknown.

How to Respond

- Run `kubectl get nodes` to check node health
- SSH into the affected node and inspect the kubelet: `systemctl status kubelet`
- Check node conditions: `kubectl describe node <node-name>`
- If the node is truly lost, drain and delete it to trigger pod rescheduling on healthy nodes
- Use `kubectl get events --all-namespaces` for cluster-wide signals

Restart Policies: Controlling Pod Recovery

The `restartPolicy` field in a pod spec governs what happens when a container exits — whether from success or failure. Choosing the right policy is critical for batch jobs, long-running services, and one-shot tasks.

Always

Restarts the container regardless of exit code. Default for Deployment, DaemonSet, and StatefulSet pods. Suitable for long-running services that should never stop.

OnFailure

Restarts only if the exit code is non-zero. Ideal for Job workloads where you want retry-on-failure but not restart-on-success behavior.

Never

Never restarts the container regardless of outcome. Used for one-shot pods, debugging containers, and strict batch jobs where you want to inspect the final state manually.

Init Containers: Prerequisites Before Your App Starts

Init containers run and complete *before* any app containers in a pod start. They are defined in `spec.initContainers` and execute sequentially — each must succeed before the next begins. If any init container fails, Kubernetes restarts it according to the pod's restart policy, holding the pod in the **Init** sub-state of Pending. Use cases include: waiting for a database to become ready, pre-populating a shared volume, or running migrations.

Pod Readiness & Liveness Probes

Kubernetes uses probes to make intelligent decisions about pod health. Configuring them correctly prevents traffic from reaching unhealthy pods and enables automatic recovery from application-level failures.

Liveness Probe

Checks whether the container is still alive. If it fails, Kubernetes **kills and restarts** the container. Use for detecting deadlocks or frozen processes that won't exit on their own.

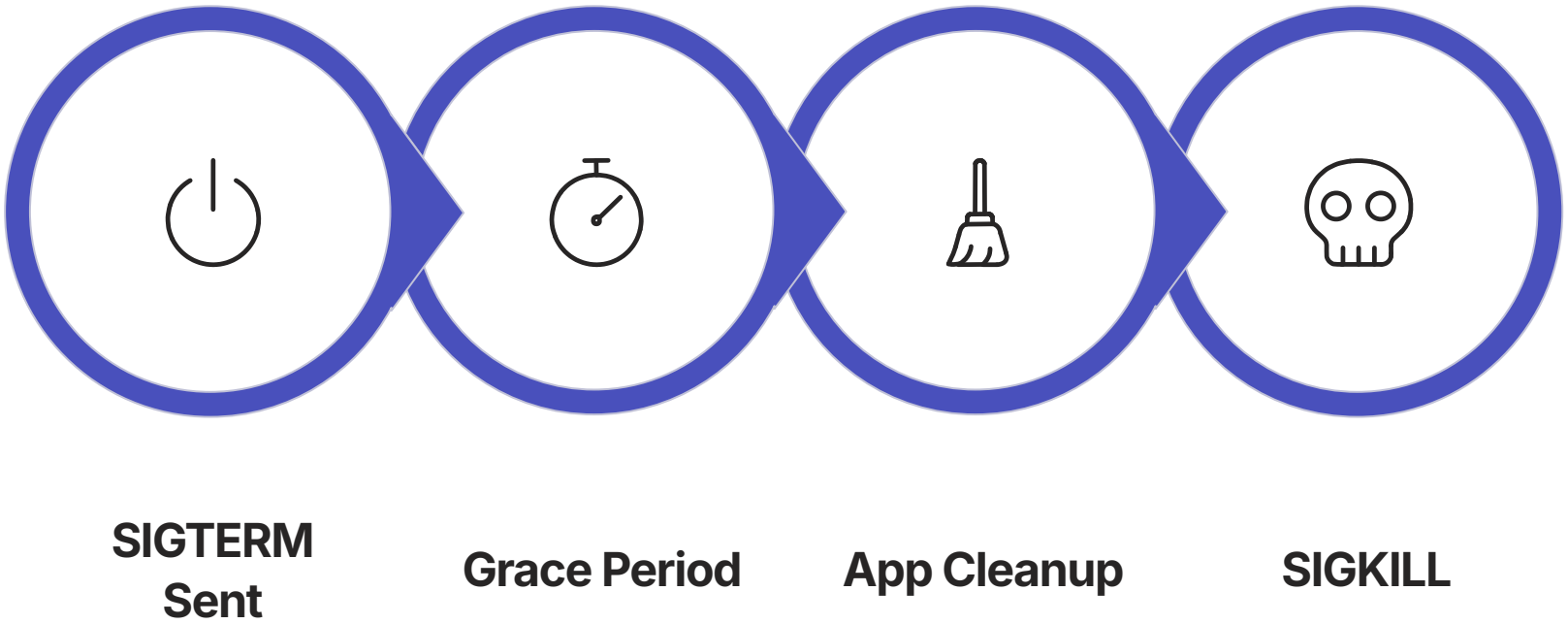
Readiness Probe

Checks whether the container is ready to serve traffic. If it fails, the pod is **removed from Service endpoints** but not restarted. Use for apps with slow startup times or dependency warm-up.

Startup Probe

Gives slow-starting containers time to initialize before liveness checks begin. Disables liveness and readiness probes until it succeeds — preventing premature restarts during heavy startup operations.

Pod Termination: The Graceful Shutdown Sequence



When a pod is deleted, Kubernetes does not kill it immediately. It sends `SIGTERM` to all containers and starts a configurable **grace period** (default: 30 seconds). This window allows your application to finish in-flight requests, flush buffers, and close connections cleanly. If the process has not exited by the end of the grace period, Kubernetes sends `SIGKILL`. Configure `terminationGracePeriodSeconds` in your pod spec to match your application's shutdown needs.

CrashLoopBackOff: The Most Common Problem

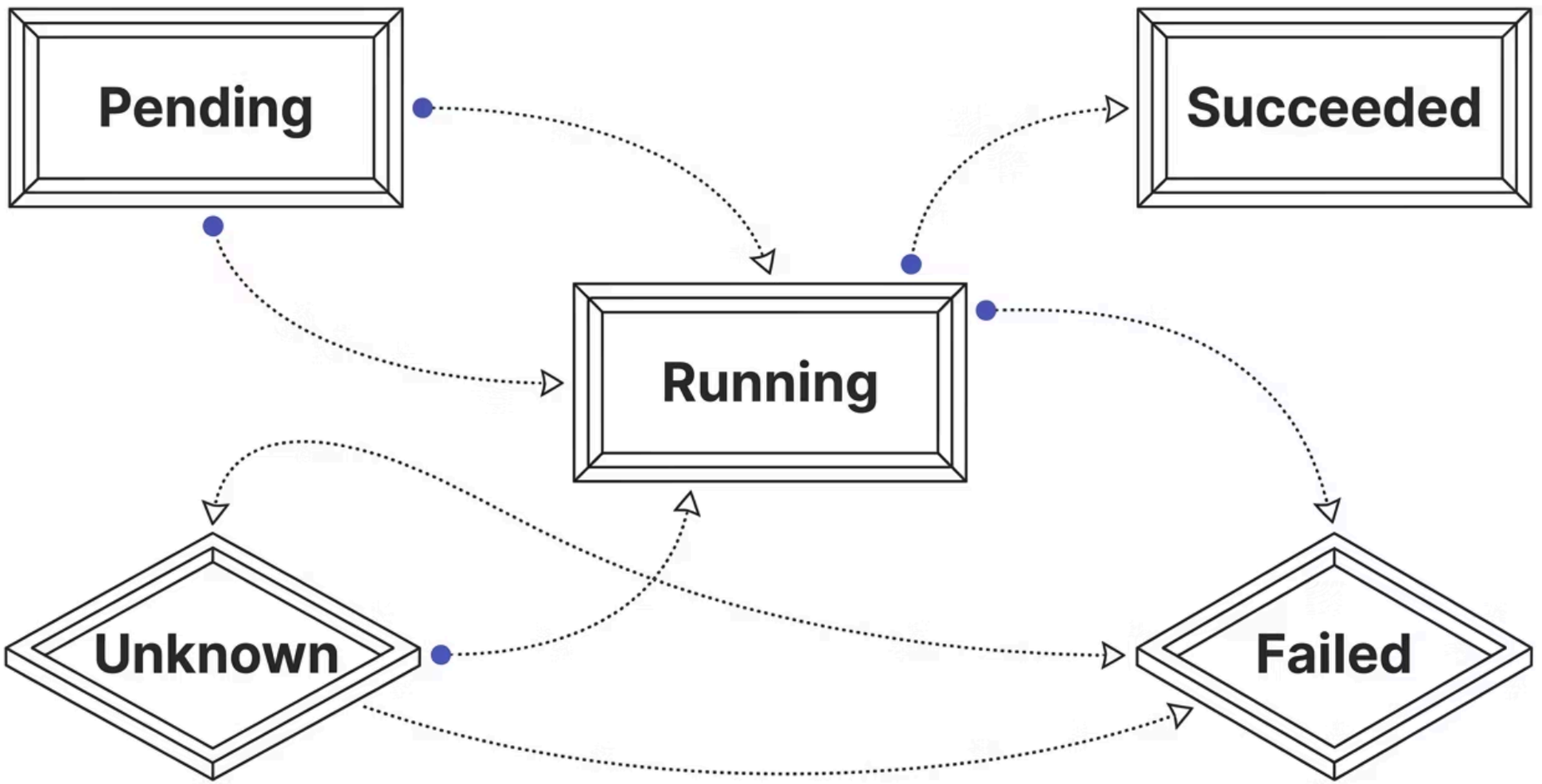
What It Is

`CrashLoopBackOff` is not a pod phase — it's a container **waiting reason**. It appears when a container repeatedly crashes and Kubernetes applies exponential back-off between restart attempts (10s, 20s, 40s... up to 5 minutes). The pod stays in Running phase but is effectively non-functional.

How to Debug It

- `kubectl logs <pod> --previous` — see logs from the crashed container instance
- `kubectl describe pod <pod>` — check `Last State` and exit code
- Look for `OOMKilled` — increase memory limits
- Check for missing environment variables or misconfigured secrets
- Verify the container `command` and `args` are correct
- Use `kubectl exec` on a debug sidecar to inspect the filesystem

Pod Phase Transitions: Full State Machine



Not all phase transitions are possible. Succeeded and Failed are **terminal** — a pod that reaches either state will not return to Running without being recreated. Unknown is transient and resolves once the node recovers or is declared lost by the control plane.

Observability: Key Commands for Every Phase

A concise reference for the commands you'll use most when monitoring pod lifecycle in production.

Situation	Command	What to Look For
Pod stuck in Pending	`kubectl describe pod <name>`	FailedScheduling events, resource constraints
Container crashing	`kubectl logs <pod> --previous`	Stack traces, error messages, exit reasons
Pod in Unknown	`kubectl describe node <node>`	Node conditions, kubelet status
Slow startup	`kubectl get events -n <ns>`	ImagePullBackOff, init container failures
Readiness failures	`kubectl get endpoints <svc>`	Missing pod IPs indicate readiness probe failures
Resource exhaustion	`kubectl top pod`	CPU throttling, memory near limits

Best Practices for Healthy Pod Lifecycles

Always Set Resource Requests and Limits

Prevents pods from being evicted unexpectedly and ensures the scheduler can make accurate placement decisions. Missing requests lead to over-committed nodes and cascading failures.

Handle SIGTERM in Your Application

Ensure your app catches SIGTERM and shuts down gracefully within `terminationGracePeriodSeconds`. Ignoring SIGTERM leads to dropped connections and data corruption during rolling updates.

Configure All Three Probe Types

Use startup probes for slow-starting apps, liveness probes for deadlock detection, and readiness probes to control traffic routing. Misconfigured probes are a top cause of unnecessary restarts.

Use Jobs and CronJobs for Finite Tasks

Don't deploy batch workloads as Deployments. Use `Job` with `restartPolicy: OnFailure` for retryable tasks and `CronJob` for scheduled work to take advantage of proper lifecycle semantics.

Key Takeaways

5

Pod Phases

Pending, Running, Succeeded, Failed, Unknown — each with distinct behavior and recovery paths.

3

Restart Policies

Always, OnFailure, and Never — match the policy to the workload type for predictable behavior.

3

Probe Types

Liveness, Readiness, and Startup probes give Kubernetes the signals it needs to route traffic and recover containers.

30

Second Grace Period

Default termination window — tune `terminationGracePeriodSeconds` to match your app's shutdown time.

Master pod lifecycle phases and you master Kubernetes operations. Every deployment strategy, autoscaling decision, and incident response flows from understanding how pods move through these states.